WEEK 06

WEB SECURITY & VULNERABILITY TESTING

STEP-BY-STEP VAPT PRACTICAL REPORT

LAB TARGET DVWA on 127.0.0.1:8082	**TOOLS** Docker • Burp Suite CE • OWASP ZAP
TESTS SQL Injection • Reflected/Stored XSS	**EVIDENCE** 7 selected screenshots

This report documents a controlled learning assessment against a deliberately vulnerable local application. Findings apply to the lab configuration only and are not a certification of a production website.

Professional submission edition • October 2026

1. Executive Summary

This practical exercise followed a basic VAPT workflow: prepare an isolated target, run controlled manual tests, inspect HTTP traffic, review scanner alerts, preserve selected evidence, and recommend fixes. Testing was performed against DVWA at `http://127.0.0.1:8082` only.

The supplied evidence supports a SQL Injection demonstration in DVWA and a Reflected XSS browser alert. Stored XSS guestbook activity was captured; the report distinguishes stored content from proof of later script execution. Burp Suite was used to inspect HTTP history, and OWASP ZAP was used to review scanner findings and an HTML report.

2. Scope and Lab Safety

In scope: local DVWA instance, test inputs entered manually, HTTP requests generated by the browser, and ZAP alerts for the same local target.
Out of scope: public websites, third-party accounts, production systems, or any target without explicit authorization.

Safety rule

Keep DVWA bound to localhost/127.0.0.1. Do not expose the intentionally vulnerable container to the public internet. Only test systems you own or have written permission to assess.

3. Tools and Their Purpose

Tool	Purpose
Docker + DVWA	Run the intentionally vulnerable application locally.
Burp Suite Community Edition	Inspect HTTP history, requests and responses.
OWASP ZAP 2.17.0	Review scanner alerts and export the HTML report.

STEP 1 — Prepare the DVWA lab

Goal: Start the local container and confirm the target is reachable.

1. Start Docker in Kali Linux and check the container with `sudo docker ps`.

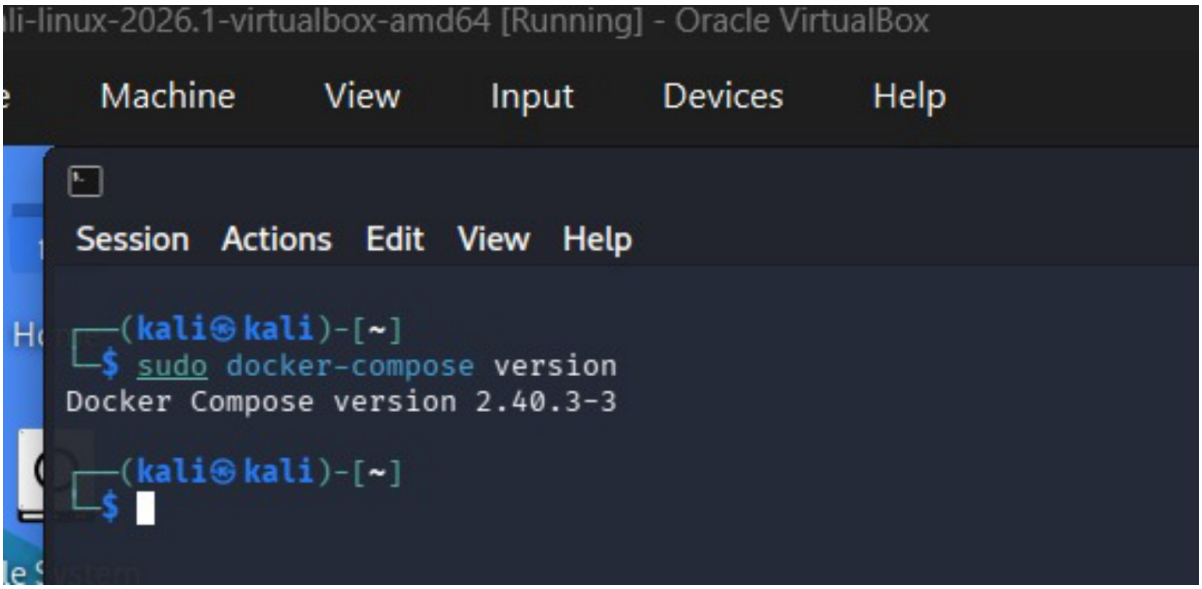
2. Open `http://127.0.0.1:8082` in the Kali browser.

3. Log in to DVWA and initialize the database if its setup page requests it.

4. For the introductory exercise, select DVWA Security = Low.

What to record: The target loads in the browser and the container is running.

Security lesson / mitigation: Use loopback-only port binding. Do not expose DVWA outside the lab.



Step 1 — Docker environment

STEP 2 — Test SQL Injection

Goal: Compare normal input with a controlled test value in DVWA's SQL Injection module.

1. Navigate to Vulnerabilities → SQL Injection.

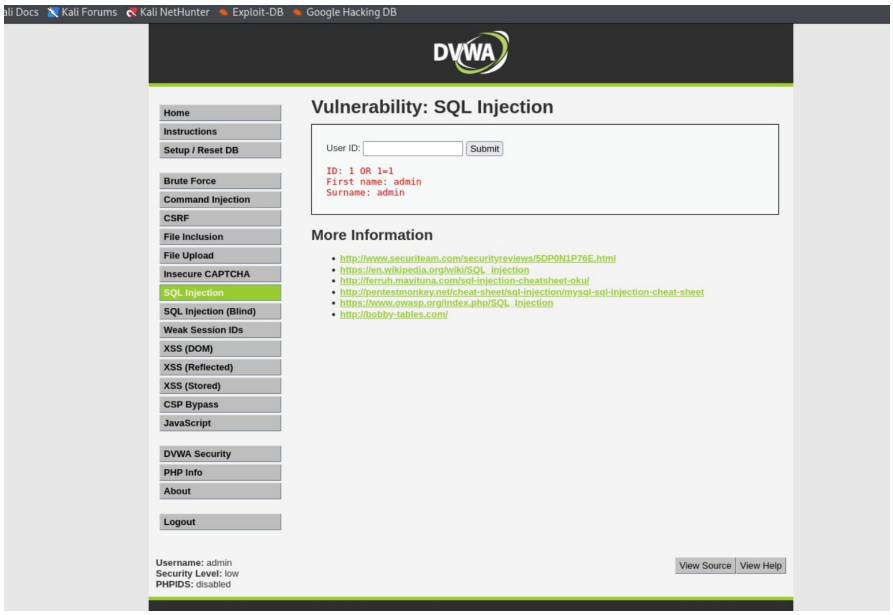
2. Submit a baseline User ID such as `1` and note the normal output.

3. In this local lab only, submit `1' OR '1'='1`.

4. Compare the returned records and capture the result screen.

What to record: The supplied screenshot shows the manipulated input affecting the response. This demonstrates SQL injection behavior in the deliberately vulnerable DVWA instance.

Security lesson / mitigation: Use prepared statements/parameterized queries, least-privilege database accounts, and safe error handling in real applications.



Step 2 — SQL Injection in DVWA

STEP 3 — Inspect HTTP traffic in Burp Suite

Goal: Find the DVWA request and inspect its request and response panes.

1. Open Burp Suite Community Edition.

2. Generate a request by using the DVWA SQL Injection module in the browser.

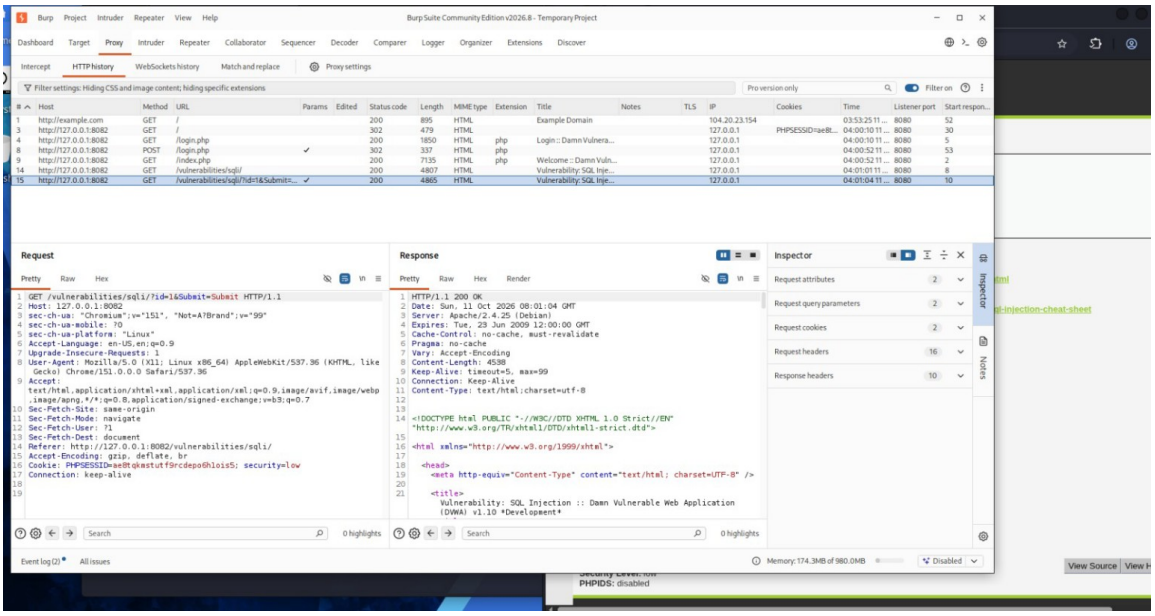
3. Select Proxy → HTTP history.

4. Select the request to the DVWA SQL Injection path and inspect the Request and Response panes.

5. Capture the request and response. Only state a payload was captured if it is visible in the request.

What to record: The selected screenshot shows a DVWA SQL Injection request and HTTP response in Burp.

Security lesson / mitigation: Burp traffic inspection is evidence of methodology; a history row alone does not prove a particular payload was sent.



Step 3 — Burp Suite HTTP history

STEP 4 — Test Reflected XSS

Goal: Check whether the response executes untrusted input as browser script.

1. Navigate to Vulnerabilities → XSS (Reflected).

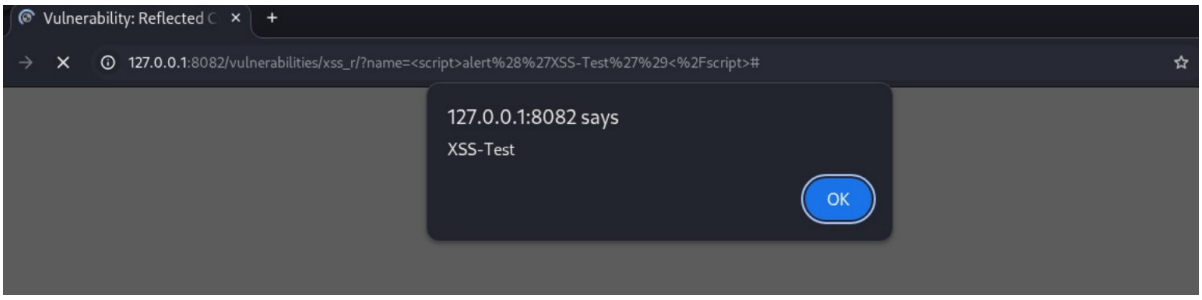
2. Submit a harmless baseline value first.

3. In the local lab, use the training payload `<script>alert(1)</script>`.

4. If a browser alert appears, capture the alert and the page URL. Dismiss the alert afterward.

What to record: The selected browser screenshot shows a JavaScript alert from 127.0.0.1:8082, supporting Reflected XSS execution in DVWA.

Security lesson / mitigation: Encode output for its HTML/JavaScript context, use safe templates, validate input as defense-in-depth, and consider CSP as an additional layer.



Step 4 — Reflected XSS execution

STEP 5 — Test Stored XSS

Goal: Determine whether submitted guestbook content persists and whether it executes.

1. Navigate to Vulnerabilities → XSS (Stored).

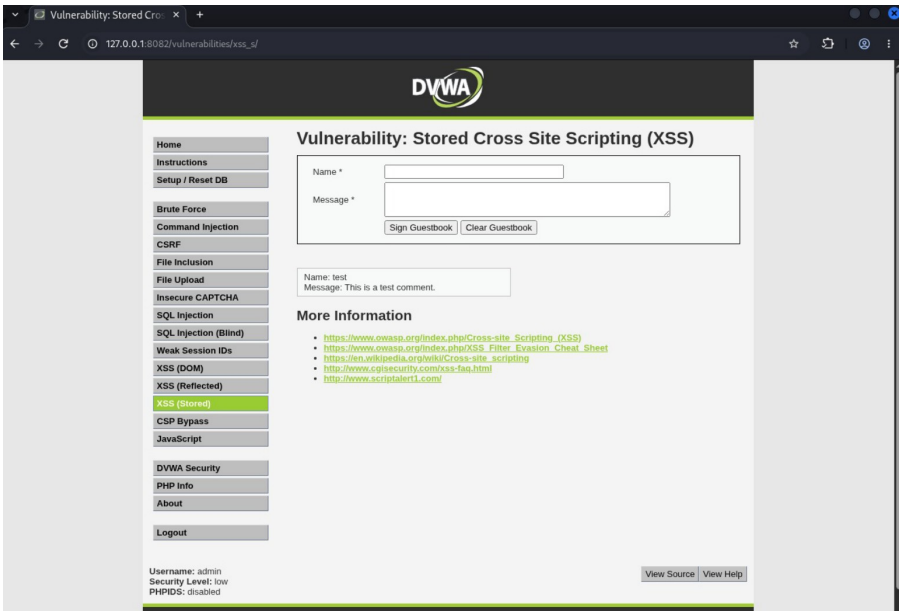
2. Submit a clearly labelled test guestbook entry.

3. Refresh the page and check whether the entry remains visible.

4. Capture stored-content evidence separately from any JavaScript alert evidence.

What to record: The screenshot supports the stored guestbook interaction. Persistence alone does not prove script execution; use the alert screenshot only if it clearly shows execution.

Security lesson / mitigation: Encode output when rendering stored content. If rich HTML is intentionally allowed, sanitize it with a well-maintained sanitizer.



Step 5 — Stored XSS guestbook

STEP 6 — Run and review OWASP ZAP

Goal: Scan only the local DVWA target and interpret the alert list carefully.

1. Launch OWASP ZAP 2.17.0.

2. Set the target to `http://127.0.0.1:8082`.

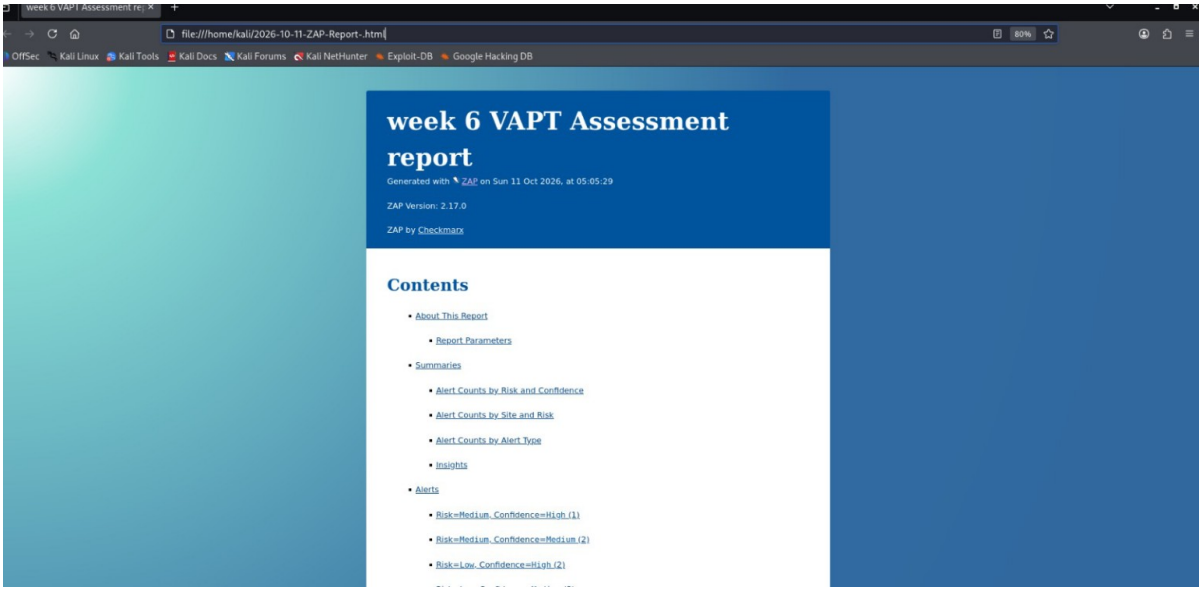
3. Run the scan and open the Alerts panel.

4. Review each alert; distinguish security weaknesses from informational or diagnostic alerts.

5. Export the HTML report and keep the original file with the project.

What to record: The selected screenshots show ZAP open and the generated assessment report. Previously recorded alert categories included directory browsing, missing security headers and cookie-attribute observations.

Security lesson / mitigation: Disable directory listing where not required, review cookie flags and security headers, and remove unnecessary server-version disclosures. Verify alerts manually.



Step 7 — ZAP report

7. Findings Summary

Finding	Evidence status	Recommended fix
SQL Injection	Demonstrated in local DVWA	Parameterized queries; least privilege; safe errors
Reflected XSS	Browser alert evidence available	Context-aware output encoding; safe templating; CSP defense-in-depth
Stored XSS	Guestbook content stored/displayed	Encode output; sanitize only where rich HTML is allowed
ZAP: Directory Browsing	Scanner observation	Disable directory listing and review exposed files
ZAP: Missing headers / cookie attributes	Scanner observations	Review CSP, anti-clickjacking, nosniff, HttpOnly, Secure and SameSite
ZAP: Authentication/session identification	Informational unless verified otherwise	Manually review session rotation, expiry, logout and authentication controls

Risk labels are qualitative and contextual, not CVSS scores. A scanner alert is a lead for manual verification, not automatic proof of exploitability.

8. OWASP Topic Notes

SQL Injection: untrusted input changes SQL query structure or logic. Prevent it primarily with parameterized queries/prepared statements and least-privilege database accounts.

Cross-Site Scripting: browser treats untrusted content as executable script. Reflected XSS appears in the immediate response; Stored XSS persists and is rendered later. Context-aware output encoding is the primary defense.

Identification and Authentication Failures: weak login/session controls may permit unauthorized access. Use secure password hashing, rate limiting, MFA where appropriate, session rotation/expiry and logout invalidation.

Security Misconfiguration: unsafe defaults, directory listing, missing headers and excess server detail increase risk. Apply secure baselines, patch software and review configuration.

OWASP category labels vary by edition. In OWASP Top 10:2021, SQLi and XSS are under A03 Injection; Security Misconfiguration is A05; the former “Broken Authentication” wording maps to A07 Identification and Authentication Failures.

9. Submission Checklist and References

■ PDF report opens and is readable
■ PowerPoint opens and all slides are readable
■ Original selected screenshots are saved separately
■ Original ZAP HTML report is retained
■ Four OWASP notes are included if required separately
■ GitHub repository contains report, presentation, notes and selected evidence

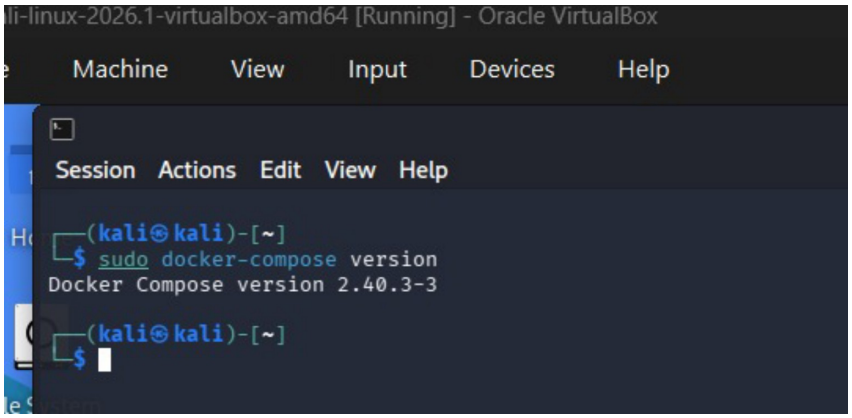
References

• OWASP Top 10: https://owasp.org/www-project-top-ten/
• OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
• OWASP Cross Site Scripting Prevention Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
• OWASP ZAP documentation: https://www.zaproxy.org/docs/
• PortSwigger Web Security Academy: https://portswigger.net/web-security

10. Evidence Index — Selected Screenshots Only

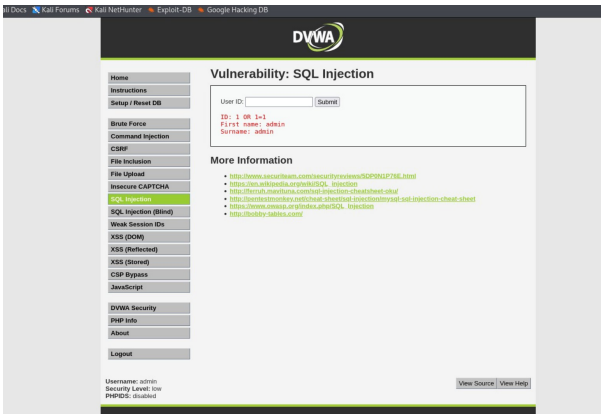
Only the screenshots most directly tied to the practical steps are included below. Keep the full original screenshot collection separately in case the instructor requests additional proof.

1. Step 1 — Docker environment



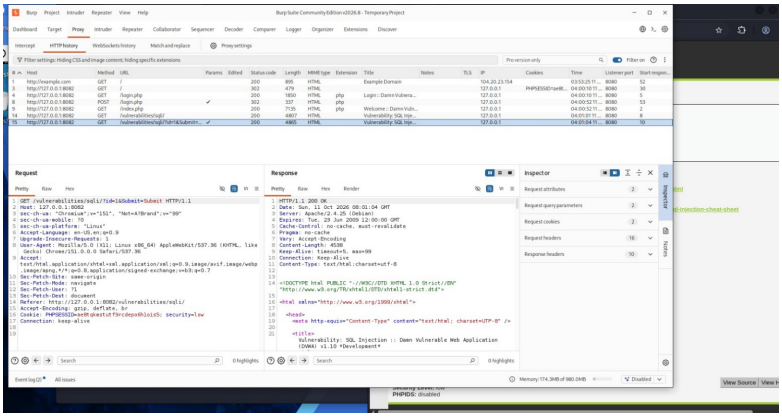
Terminal evidence of Docker Compose availability / lab preparation.

2. Step 2 — SQL Injection in DVWA



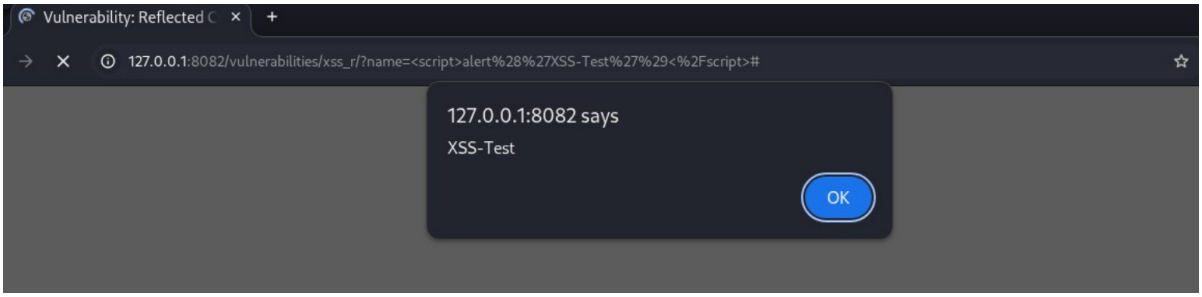
DVWA SQL Injection module showing manipulated input and the application response.

3. Step 3 — Burp Suite HTTP history



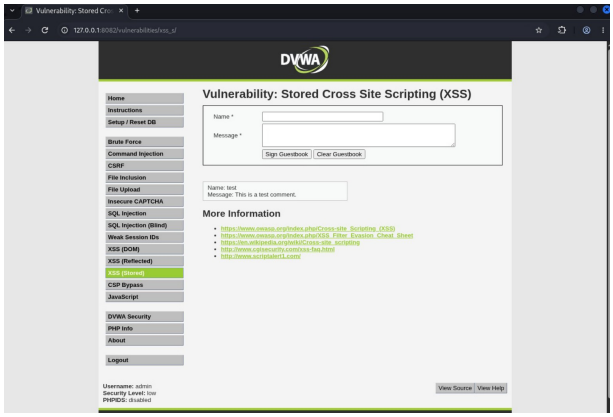
Burp HTTP history with the DVWA SQL Injection request selected; request/response panes are visible.

4. Step 4 — Reflected XSS execution



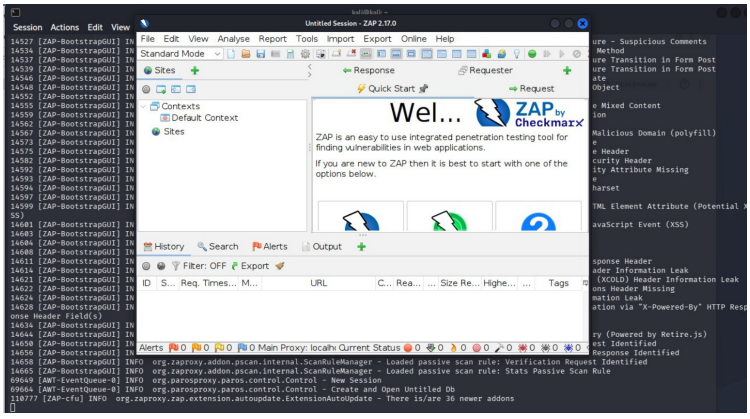
Browser alert from the local DVWA target, supporting reflected JavaScript execution.

5. Step 5 — Stored XSS guestbook



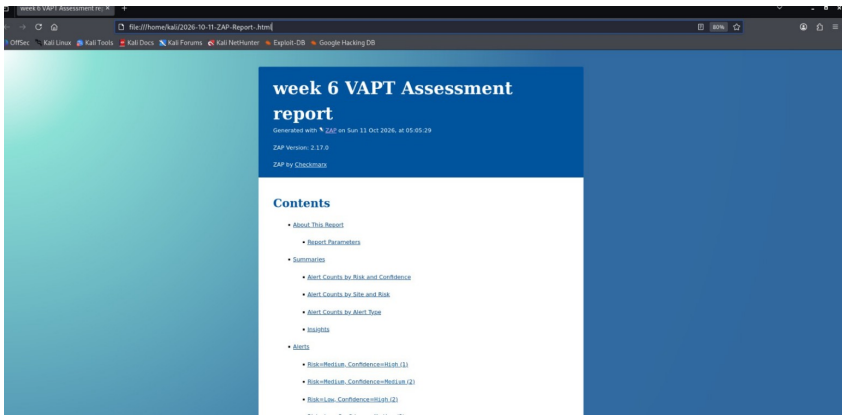
Stored XSS module / guestbook evidence. This supports the stored-content test; it does not alone prove execution.

6. Step 6 — ZAP setup



OWASP ZAP application opened for the local assessment.

7. Step 7 — ZAP report



ZAP assessment report page captured as evidence.